



SwarmProtocol

A 3D real-time survivor-style game built around orthogonal architecture and physics-free combat.

CMPE 485 · Term Project · Option B

Alperen Garip

Unity 6.3 URP · macOS Standalone

What is SwarmProtocol?

And what's actually distinct from Vampire Survivors

The game

- 3D survivor-style arena: auto-firing weapons, level up between waves, fight bosses, collect treasure
- Survive timed stages against escalating swarms; pick passive items + weapons to grow stronger

What's distinct from VS

- 3D arena with orbiting Cinemachine camera — VS is 2D top-down sprites
- Slot-machine-style reveal animation for treasure-chest rewards
- Built as a CMPE 485 case study in orthogonal architecture — adding new enemies / weapons / passives is data work, not engine code

Design

Mechanics + key architectural choices

GOAL

Survive 5 timed stages, level up between waves

LOSE

Player HP reaches 0 → Game Over screen

WIN

Stage 5 timer expires → Victory screen

State coordination

GameManager FSM + EventBus

vs. god-class manager

Each system subscribes to OnGameStateChanged. New state-aware UI = 1 file, no manager edits.

Enemy / weapon definition

Strategy-as-ScriptableObject

vs. one MonoBehaviour per type

New enemy = EnemyBehaviorSO subclass + asset. New weapon = FireStrategySO subclass. Data-driven content.

Stat stacking (passives)

Decorator chain (IStatProvider)

vs. flat dictionary sums

New stat = new StatType enum value. Curse / Might / Area / Recovery all use the same plumbing automatically.

Libraries: Unity 6.3 URP · Cinemachine · Input System · TextMeshPro · Unity AI (sound + sprite generation)

Technical Challenges

Three problems & how the architecture handled them

01

Aura/melee detection at scale

Why hard: `Naïve Physics.OverlapSphereNonAlloc` per weapon per tick scales poorly with hundreds of enemies and depends on collision-matrix configuration.

Solution: `ActiveEnemyRegistry` + `SqrMagnitude` — cached enemy list, $O(N)$ scan, zero physics queries.

Alternative: `Physics.OverlapSphereNonAlloc` — rejected: relies on collision-matrix tuning + spatial broadphase we don't need (no static obstacles in arena). Benchmarked vs registry — slide 5b.

02

Adding content without touching engine code

Why hard: Hierarchy of one `MonoBehaviour` per enemy/weapon type couples engineering to content; every new asset needs a new script + recompile.

Solution: `Strategy-as-ScriptableObject` — one generic host (`BehaviorDrivenEnemy` / `StrategyWeapon`) executes a SO asset. New type = new SO + asset.

Alternative: Inheritance hierarchy — rejected: subclass count grows with content, blocks designers without programmers. Trade-off: SO subclass count grows in `/Behaviors/` but it's content-driven, not engineer-blocking.

03

Spawn/despawn churn at high rates

Why hard: `Per-shot Instantiate + Destroy` generates GC pressure that causes periodic frame spikes during heavy combat.

Solution: `ObjectPoolManager` — auto-creates pools on first `Get<T>()`. Every projectile, enemy, gem, coin implements `IPoolable`.

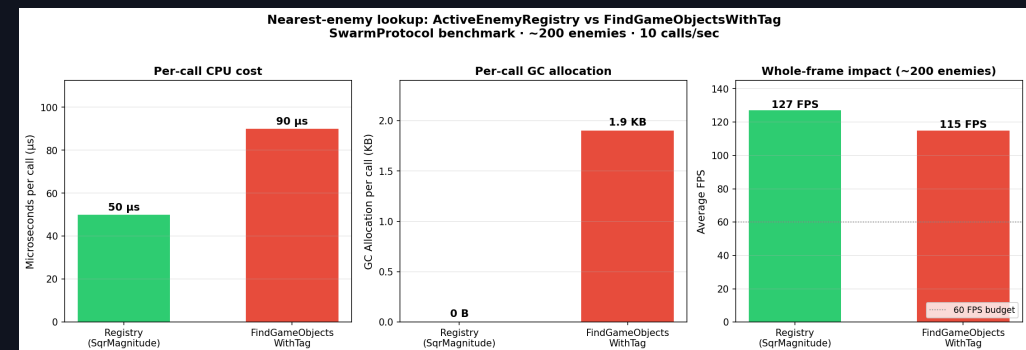
Alternative: `Per-shot Instantiate / Destroy` — rejected: `GC.Alloc` per call (~1.9 KB observed in profiler). Trade-off: pool adds `OnSpawn/OnDespawn` lifecycle states to manage.

Performance — Metrics & Method

Benchmark setup

Method

- Tools: Unity Profiler + custom ProfilerMarker rows in hot paths
- Sampling: 10 s warm-up, then 10 s averaging window per data point
- Scene state: player AFK at arena center, enemy count forced via test driver
- Two metrics: (1) avg frame time / FPS, (2) per-call cost + GC alloc
- Hardware: macOS, Unity 6.3 Editor (Editor adds ~30% overhead vs build)

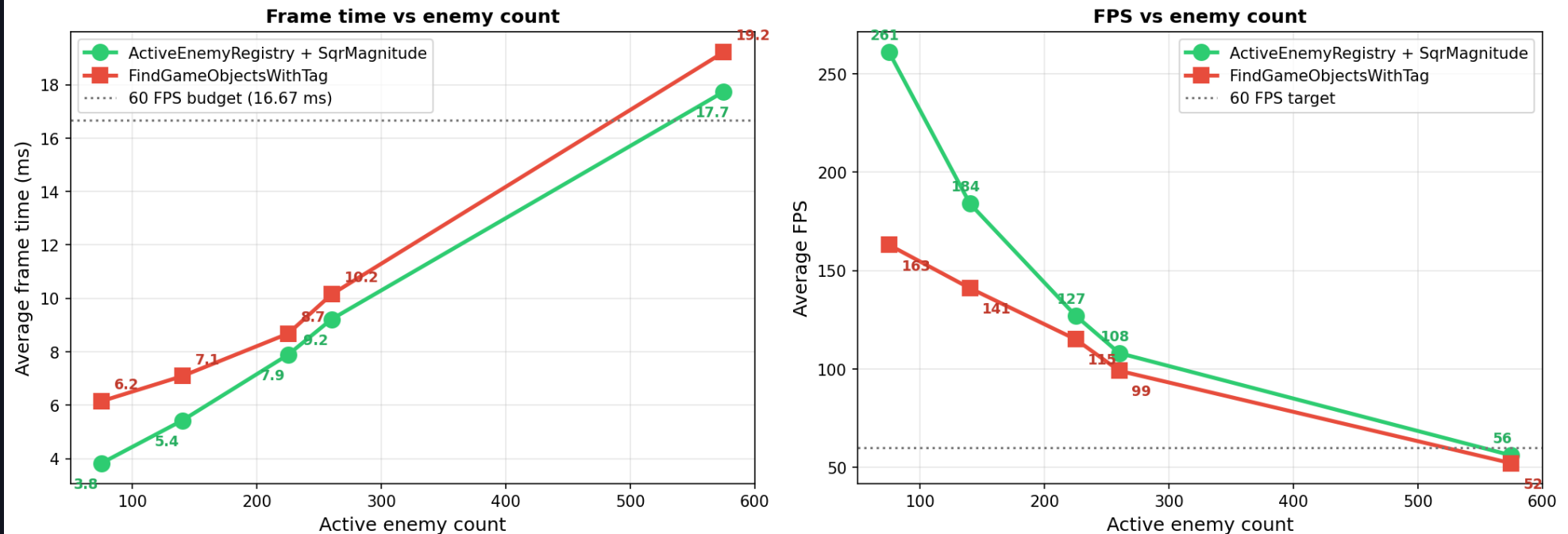


Takeaway: Registry is 1.8× faster per call AND eliminates 19 KB/s of GC pressure that drives p99 frame spikes.

Performance — Interaction Figure

Lookup method × enemy count (X = enemies, two curves, Y = perf)

Nearest-enemy lookup method × enemy count interaction
SwarmProtocol benchmark · Unity 6.3 URP · 600-frame avg per point



Registry consistently wins across 75 → 575 enemies. 60 FPS budget crossed earlier with FindWithTag.

Gameplay

30-second clip



Watch the gameplay clip

https://drive.google.com/file/d/12JRbtkcfVK8df7319K_opKi1g20CGa4V/view?usp=drive_link

Hosted on Google Drive · click the link above (or paste into a browser)

Conclusion

What we learned & what comes next

What I learned

- Orthogonal design pays off — adding 5 missing passive consumers took one afternoon, no combat-code edits.
- Strategy-as-SO + Decorator chain made content additions cheap and refactor-safe.
- Profiler-driven decisions: the registry vs FindWithTag win was 1.8× per call AND avoided 19 KB/s of GC.
- ObjectPool + IPoolable removed Instantiate spikes during heavy combat.

With more time

- Migrate remaining legacy weapons (RapidFireWeapon, ShotgunWeapon) to StrategyWeapon + SO.
- Replace placeholder art with rigged 3D characters + animations.
- Multiplayer co-op via Mirror — would test our event architecture under network sync stress.
- Run profiler challenges 2 + 3 (chest VFX render load, projectile physics scaling).
- Save game / persistent run state across application restarts.

Thanks — questions?